GPL (Game Play Language) Reference

GPL is a custom scripting language, used to implement a majority of the AI code in Majesty. The language is function based and has a simple grammar.

Basic syntax

The language is not case sensitive, so begin and Begin are equivalent. Statements are terminated with a semi-colon ; and comments can be added to the source code using //, all text from that point, to the end of the line will be ignored by the compiler.

Keywords

and
begin
break
continue
declare
define
else
end
expression
false
foreach
function
if
prototypep
return
true
while

Types

GPL recognizes a set of intrinsic types, they are:

agent
agentref
boolean
coordinate
float
function

integer
list
string

Boolean

Can contain a true or false value.

Coordinate

Contains an X/Y coordinate on the game map. Coordinates can be compared for equality directly. The members of the coordinate can be extracted with the built-in functions $GetX and $GetY.

Float

The float type is in fact, a fixed-point numeric value, with the maximum signed value integer portion of 2^{23} and a maximum fixed-point precision of 1/255. The reason the float value is defined as fixed-point is that the GPL logic is run in parallel on client machines in a multiplayer game. The floating point units on different clients could produce differing rounding errors, which would cause the clients to go out of sync.

Integer

A signed value with the range of 2^{31}.

List

A single dimensional list, whose members can be of any intrinsic type.

String

A string of characters.

Agent

An agent is a GPL structure containing pre-defined attributes as well as a link to a object in the game world, such as a hero or a building. An agent's attributes are defined by making a prototype, which is analogous to a C++ structure. The prototypes can then be used to create pre-defined types of agents in a GPL data file.

AgentRef

This is a reference to an agent, not the agent itself. This is also referred to as an Agent Number.

Function

A named block of GPL code with optional input parameters and a return type.

Functions are defined as follows

```
function FunctionName(Function Parameters) is [Function Return Type]
declare
     Local FunctionValues
begin
     Function Body
end
```

Example: Print hello to the output if the input agent is a Rogue.

```
function HelloRogue(agent thisAgent) is boolean
declare
     integer I;
begin

end
```

Once a function is defined, it can be called or referenced by its name, preceded by a $.

Example: To call the function from the previous example.

```
$HelloRogue(theRogue);
```

Expressions

Expressions in GPL are used to define integer values as an easier to understand/read text. This is equivalent to defining a constant value in other languages. To define an expression, you use the expression keyword, followed by a #, then the text that will be equivalent to the integer value.

Ex.

```
// Number of heroes that have to die before a Graveyard is created
expression #Graveyard_limit 15
```

You can then use the expression anywhere else in the GPL code as a replacement for the integer value.

Ex.

```
if (palace's "num_deceased_heroes" >= #Graveyard_limit)
```

It is best practices to use expressions for values, rather than just specifying the number. This is more readable and easier to change.

The GPL code for the base game has defined many constant values in globals.gpl and mx_globals.gpl

Unlike truly compiled languages, such as C++, the expressions are not resolved at compile time and are not validated to have been pre-defined, they are resolved at load time. This is allows expression values to be defined by external data, such as the engine-side attribute system. The attribute system will look at the expression name and if it begins with ATTRIB_, it will remove that prefix and replace the value with the matching attribute's ID.

Ex.

```
$setattribute ( thisagent, #ATTRIB_FirstStageBuilt, 1 );
```

On load of the GPL bytecode, ATTRIB_FirstStateBuilt will be replaced with the FirstStageBuilt ID.

Agent Attributes

The Agent's attributes are defined by the Agent's prototype and its starting values are defined by its definition. Additional attributes can be added to the Agent by calling AddAttribute or an attribute can be remove by calling RemoveAttribute. Accessing an Agent's attribute is done by using the Agent's possessive operator, which is simply an 's on the end of the Agents instance.

Ex.

```
iDeceasedHeroes = palace's "num_deceased_heroes";

palace's "num_deceased_heroes" = iDeceasedHeroes;
```

The first line is accessing the “num_deceased_heroes” attribute and assigning it to the variable iDeceasedHeroes. The second line is assigning iDeceasedHeroes to the agent's "num_deceased_heroes" attribute.

An error will be raised if code tries to access an attribute that an agent does not have. If you are unsure if an agent will have the attribute defined, you should check to see if the attribute exists before accessing it.

Ex.

```
if ($HasAttribute("num_deceased_heroes", palace))
begin
     iDeceasedHeroes = palace's "num_deceased_heroes";
end
```

Custom attributes that are not part of the Agent's protytype/definition and be added with the AddAttribute function.

Ex.

```
// Get the root agent and put a new integer attribute on it
// defaulting the value to 500
     AIRootAgent = $RetrieveAgent ( "GplAIRoot" );
     $AddAttribute(AIRootAgent, "MyNewAttribute", "integer", 500);
```

Unit Attributes

Any Agent that is displayed in the game, is bound to a game engine object called a Unit. The engine-side Unit object contains the definition for what image is displayed for the Unit and controls the animation, movement and other UI aspects of the Unit/Agent pair. The Agent controls its bound Unit through GPL function calls and through the Unit's attribute system. The Unit's attribute system is accessed through the GetAttribute, SetAttribute functions. The Unit's attributes are used to hold information that both the GPL Agent and the engine-side Unit need to access. This includes the current hit points of the Agent/Unit, its experience level, its sight range, etc. All engine-side attributes are integers only.

GPL code cannot currently add any new engine-side attributes. This is not necessarily a problem, as the GPL side attributes are more flexible because of their simpler syntax and that they can hold any valid GPL type.

Engine side attributes are referenced with the #ATTRIB_Name expression, where Name is the name of the attribute. Ex. #ATTRIB_Intelligence is the Unit's intelligence attribute.

Ex.

```
$setattribute ( thisagent, #ATTRIB_Intelligence, 10 );

myIntelligence = $getattribute( thisagent, #ATTRIB_Intelligence );
```

Built-in functions

Various functions are defined in C++ by the game engine and are exposed to GPL. These functions are accessed just like functions that are defined in a GPL script.

Example: Call the built-in fuction IsMoving to test to see if an agent is moving and if it is, call the built-in function StopMoving to stop it.

```
if ( $IsMoving ( ThisAgent ) == TRUE )
begin
     $StopMoving( ThisAgent);
end
```

Alphabetical List of Functions

AddAttribute
AddLists
AdjustAttribute
AdjustPlayerData
AgentHasInventoryInItem
AgentNumber
BreakPoint
BuildingIsRecruiting
CancelGuildMembership
CanDropInventoryItem
CastSpell
ChangeUnitType
CheckEffector
ClearDrawEffects
ClearEngineDeathFlags
ClearFlag
ClearList
CombatSignal
Concatenate
CountInventoryItem
CreateAgent
CreateEffector
CreateMissile
CreateNewInventoryItem
CreateSpellUnit
DebugOut
DeclareLoss
DeclareVictory

DeleteAgent
DeleteAllEffectors
DeleteEffector
DeleteGamePiece
DeleteInventoryItem
DisableUnitType
DistanceBetweenAgents
DistanceBetweenCoords
DwarvesVoice_SetOperative
ElvesVoice_SetOperative
EnableUnitType
EnchantWizTower
FadeIn
FadeOut
FillSpecialLairList
FindInventoryItem
ForgetSpell
GetAttribute
GetBestSpell
GetBoardExtents
GetBuildingContainer
GetBuildingContents
GetContainerAgent
GetEntranceLoc
GetFlag
GetInventoryItemAgentType
GetInventoryItemDropType
GetInventoryItems
GetNearestHiddenCoord
GetNextWayPoint
GetParent
GetPlayerData
GetPlayerTeamNumber
GetSpecialEvent1Script
GetSpecialEvent2Script
GetSpellAttribute
GetUnitPlayerNumber
GetVictoryConditionIndex
GetVictoryConditionModifier
GetX
GetY
Halt
HasAttribute
HasSpeedTrail
HasWayPoints
Hide

IncrementStatCounter
InsideBuilding
IsAdjacent
IsEnteringBuilding
IsHidden
IsInventoryItem
IsMessageFlagPresent
IsMoving
IsRunning
IsSpellAvailable
IsValidGamePiece
KillThread
LastMusicTrack
LearnSpell
ListMember
ListPalaces
ListObjects
ListSize
ListTitles
LocalChatMessage
LocationOf
LookupFunction
MagicalAdjustAttribute
MakeCoord
MakeInventoryAttribute
MessageFlag
MiniMapAnimation
Move
NeutralTeamNumber
NewTeamNumber
NewThread
NoDebugMessages
NullAgent
Parent
PathCost
PerformAction
PlaceRewardFlag
PlayMusic
PlaySound
RandomCoord
RandomNumber
RemoveAttribute
RemoveBuildingLimit
RemoveListMember
RemoveTitles
ReRandom

ResumeThread
RetrieveAgent
RevealArea
RevealWholeMap
RevertUnitType
RunThread
RunThreadOnce
SetAttribute
SetBuildingLimit
SetDrawEffects
SetEffectorDirection
SetHallFlag
SetPalace
SetParent
SetPlayerTeamNumber
SetShareReveal
SetSpellUnitTimeout
SetStoppedMusicCallback
SetThreadInterval
SetTreasuryValues
SetUnitPlayerNumber
ShowMinimap
SpawnGraveyard
SpawnUnit
SpecifyIntent
SpecifyName
Sqrt
StartEarthquakeSpell
StopEarthquakeSpell
StopMoving
SuspendThread
TeleportToPoint
TeleportToUnit
TransferInventoryItem
TurnOffSpeedTrail
TurnOnSpeedTrail
Unhide
UnitsAreSamePlayer
UnitIsLocalPlayer
UnitIsOffmapLair
UpgradeAgentAttributes
ValidFunction

Functions

AddAttribute

Add a named GPL attribute. If the attribute is already attached to the agent, nothing is done and false is returned.

Parameters:

agent	The agent to check.
string	The attribute name
string	The attribute type. This can be any of the valid GPL types.
value	An optional default value the attribute will be set to if the attribute does not already exist.

Returns:

boolean	True if the attribute was added, false if it was already there.

AddLists

Merge two or more lists together.

Parameters:

list	List to merge. There can be multiple lists specified, all will be merged into a new list.

Returns:

list	The merged list.

AdjustAttribute

Changes an attribute value on an agent by a delta.

Parameters:

agent	The agent to change.
integer	The attribute's ID.
integer	The value to add to the attribute's current value

Returns: Nothing

AdjustPlayerData

Get a value from a player.

Parameters:

agent	The agent who's player the data will be adjusted.
string	The data type. This can be "gold".
integer	Amount to adjust the value by.
String (optional)	Unit/Building type the gold is buying. Used for statistics display only. The adjustment value must be negative.

AgentHasInventoryItem

Returns true if the agent as an inventory item.

Parameters:

integer	The inventory item ID. If this is -1, then any Quest inventory item will match. If passing in -1, be sure to pass it in, in a variable. The input variable will be filled with the found inventory item id.
agent	The agent to check.

Returns:

boolean	True if the agent has the inventory item.

AgentNumber

Return the agentRef of the agent. AgentRef values are used when storing a reference to an agent in an attribute of another agent.

Parameters:

agent	The agent to retrieve the reference id from.

Returns:

agentRef	The reference ID of the agent.

BreakPoint

Break into the GPL debugger.

Parameters:

boolean or integer	If it is a boolean value and it is true, then the debugger will stop the execution.

Returns: Nothing.

BuildingIsRecruiting

Returns true if the building is in the process of recruiting a new member.

Parameters:

agent	The building agent.

Returns:

boolean	True if the building is recruiting, false if not.

CancelGuildMembership

Removes an agent from its Guild's engine-side list. The caller still needs to disconnect any links on the GPL side, such as removing the agent from it's “home's” “member” list.

Parameters:

agent	The agent to remove from its Guild.

Returns: Nothing.

CanDropInventoryItem

Returns true if the inventory item can be dropped.

Parameters:

integer or string	The inventory item type.

Returns:

boolean	True if the item can be dropped.

CastSpell

Have the agent cast a spell.

Parameters:

agent	The agent to cast the spell.
string	The spell name.

agent or coordinate	The target of the spell. The target agent can be the casting agent.

Returns: Nothing.

ChangeUnitType

Changes the type of a unit from one type to another. The original type is retained in the ATTRIB_OriginalType field. This can change only building and vehicle units. This does not update any attributes on the unit to be exactly like the new unit type, only the appearance. Use RevertUnitType to change back.

Parameters:

agent	The agent to change.
string	The name of the unit type to change to.

Returns: Nothing.

CheckEffector

Return true if the agent has the effect applied to it.

Parameters:

agent	The agent to check
string	The effect type to check for.

Returns:

boolean	True if the agent has the effect.

ClearDrawEffects

Clears any transparency or other drawing effects the unit might have on it.

Parameters:

agent	The agent to update.

Returns: Nothing.

ClearEngineDeathFlags

Clears any flags marking the unit as 'dead'. This is called when resurrecting a unit. It does not unhide the unit or reconnect any other actions/tasks.

Parameters:

agent	The agent to resurrect.

Returns: Nothing.

ClearFlag

Removes the agent from chasing any Reward Flags.

Parameters:

agent	The agent remove from chasing any Reward Flags.

Returns: Nothing

ClearList

Remove all the members of a list.

Parameters:

list	The list to clear.

Returns: Nothing

CombatSignal

Used to signal that a unit is in combat. This will show a flash on the mini-map.

Parameters:

agent	The agent that is in combat.

Returns: Nothing.

Concatenate

Add all the input parameters to a list and return the list.

Parameters:

any type	One or more parameters of any type, which will be added to a list.

Returns:

list	The input parameters in list form.

CountInventoryItem

Returns the number of instances of an inventory item type.

Parameters:

integer	The inventory item ID.

Returns:

integer	The number of instances of the inventory item.

CreateAgent

Create a new agent from a prototype. Normally you do not create agents directly, they are created for you when a new game object is created. Agents that are not attached to game objects are useful for persistent storage of game play variables however.

Parameters:

string	Agent prototype name
string	(optional) Unique name of the agent.

Return:

agent	The new agent.

CreateEffector

Add an effect to an agent. If the effect already exists, the effect's duration is just extended.

Parameters:

agent	Agent to attach the effector to.
string	The effect type. These are defined in the Overlay Descriptions.
integer	Duration in milliseconds.
integer	(optional) A number value to float up during the effect. Usually used for damage values, etc.
string	(optional) Effect option **after** — After the effect completes, the effect's DeathScript will be called. **infinite** — The effect is infinite.

Returns: Nothing

CreateMissile

Create a missile effector for the agent.

Parameters:

string	The missile type. Missiles are defined in the Overlay Descriptions.
agent	The agent that is firing the missile
coordinate or agentref	The target, which can be a coordinate or an agent reference.

Returns: Nothing

CreateNewInventoryItem

Create a new inventory item and give it to an agent.

Parameters:

integer	The item type ID.
agent	The recipient of the item.
boolean (optional)	If true, allow duplicate items. Defaults to false.

Returns: Nothing.

CreateSpellUnit

Create an spell agent. This is for spells that have a persistent agent that deals the attack/effect on the surrounding enemy agents.

Parameters:

agent	The agent that is spawning the spell.
string	The spell name.

Returns: Nothing.

DebugOut

Output the string to the debug log.

Parameters:

string	String to output.

Returns: Nothing.

DeclareLoss

Declare a loss for the player.

Parameters:

agent or integer	An agent of the losing player or the losing player's number

Returns: Nothing

DeclareVictory

Declare victory for a player.

Parameters:

agent or integer	An agent of the winning player or the winning player's number
coordinate or agent (optional)	Location or agent to look at. If not specified, the player's Palace will be used.

Returns: Nothing

DeleteAgent

Delete the agent. This does not delete the unit the agent is attached to so use with care. DeleteAgent is only called in special situations where the unit will clean up after itself, such as some buildings.

Parameters:

agent	The agent to delete.

Returns: Nothing

DeleteAllEffectors

Delete any effectors that are attached to the game object the agent is bound to.

Parameters:

agent	The agent to remove the effectors from.

Returns: Nothing

DeleteEffector

Remove an effect from an agent.

Parameters:

agent	Agent to remove the effect from.
string	The effect type to remove.

Returns: Nothing

DeleteGamePiece

Delete the game object that is associated with the agent.

Parameters:

agent	The agent to delete.

Returns: Nothing

DeleteInventoryItem

Remove an inventory item.

Parameters:

integer	The item type ID.
agent	The agent to remove the item from.

Returns: Nothing.

DisableUnitType

Prevents the specified unit type from appearing in the quest. This is used to limit what the player can build/recruit.

Parameters:

string	The name of the unit type.

Returns: Nothing.

DistanceBetweenAgents

Return the distance between two agents. The distance is in World coordinates and will be calculated from the closest footprint edges.

Parameters:

agent	Agent 1
agent	Agent 2

Return:

integer	Distance in world coordinates.

DistanceBetweenCoords

Return the distance between two coordinates.

Parameters:

coordinate	Coordinate 1
coordinate	Coordinate 2

Return:

integer	Distance in world coordinates.

DwarvesVoice_SetOperative

Sets whether or not a periodic announcement that Dwarves are available to join the kingdom is made.

Parameters:

integer	0 = disabled, 1 = enabled

Returns: Nothing

ElvesVoice_SetOperative

Sets whether or not a periodic announcement that Elves are available to join the kingdom is made.

Parameters:

integer	0 = disabled, 1 = enabled

Returns: Nothing

EnableUnitType

Allows the specified unit type to appear in the quest.

Parameters:

string	The name of the unit type.

Returns: Nothing.

EnchantWizTower

Enchant the supplied Wizard Tower.

Parameters:

agent	The wizard tower to enchant.

Returns: Nothing

FadeIn

Tell the agent to fade in over a certain amount of time.

Parameters:

agent	The agent to fade in.
integer	Milliseconds between steps of adjusting the fade, with there being 10 discreet steps in the fade. Ex. Setting this to 100 would mean that the unit would fade in over 1000 milliseconds (1 second).

Returns: Nothing

FadeOut

Tell the agent to fade out over a certain amount of time.

Parameters:

agent	The agent to fade out.
integer	Milliseconds between steps of adjusting the fade, with there being 10 discreet steps in the fade. Ex. Setting this to 100 would mean that the unit would fade out over 1000 milliseconds (1 second).

Returns: Nothing

FillSpecialLairList

Take an input list of agents and remove any agents whose 'title' attribute matches the input. The 'title' of an agent is equivalent to the unit type. The removed agents will be returned in another list. This is similar to ListTitles, except the source list is modified as well.

Parameters:

agent	A lair agent.
list	A list to fill with the types of units the lair spawns. The list will be cleared before anything is added.

Returns:

integer	The number of types the lair spawns.

FindInventoryItem

Returns the agent that is holding the inventory item.

Parameters:

integer	The inventory item ID.

Returns:

agent	The agent that has the item or null.

ForgetSpell

Have an agent forget a spell.

Parameters:

agent	The agent that will forget the spell.
string	The spell name.

Returns: Nothing.

GetAttribute

Get an engine-side attribute value from an agent.

Parameters:

agent	The agent to get the attribute from.
integer	The attribute's ID.

Returns:

integer	The attribute's value

GetBestSpell

Returns the name of the best spell for an agent, picked from the supplied list.

Parameters:

agent	The agent to check.
integer	The spell list to check 1 — Attack list. Use the #List_Attack global expression 2 — Combat utility. Use the #List_Combat_Utility global expression. 3 — Other. Use the #List_Other global expression.

Returns:

string	The spell name. Can be "nothing".

GetBoardExtents

Return the size of the map in 2D coordinates.

Parameters: None

Returns:

coordinate	The map x and y size.

GetBuildingContainer

If the supplied agent is inside a building, return a reference to the building.

Parameters:

agent	The agent to check.

Returns:

agent	The building the source agent is in or a null agent if the source agent is not in a building.

GetBuildingContents

Return all of the agents that are inside a building as a list.

Parameters:

agent	The building agent

Returns:

list	The agents inside the building.

GetContainerAgent

Return the parent agent that owns the input agent. This is usually used to get the agent that an effect agent is attached to.

Parameters:

agent	The agent to get the parent of.

Returns:

agentRef	The parent agent reference or -1

GetEntranceLoc

Return the location of the "entrance" to a building.

Parameters:

agent	The building to get the entrance location of.

Returns:

coordinate	The location of the entrance.

GetFlag

Get the supplied agent's Reward Flag parent. Use GetParent to return an agent's Guild/Spawner parent.

Parameters:

agent	The agent to get the parent of.

Returns:

agent	The parent agent.

GetInventoryItemAgentType

Get the type of agent the item should use while it is dropped on the map.

Parameters:

integer or string	The inventory item ID.

Returns:

string	The type of agent to use. This is can be passed to functions, such as SpawnUnit to create an instance of the item on the map.

GetInventoryItemDropType

Get how the inventory item should be dropped.

The different between drop type 1 and drop type 2 is that if a hero has an item and its drop type is 1, the item will remain on the hero even after it has been killed, as long as the hero is in its 'gravestone' state. When the gravestone is removed, the item will then be dropped. If the type is 2, the item is dropped immediately after the hero is killed.

Parameters:

integer or string	The inventory item ID.

Returns:

integer	The item drop type. 0 = do not drop, 1 = drop when removed from map. 2 = drop immediately when killed.

GetInventoryItems

Get a list of inventory items the agent has.

Parameters:

agent	The agent to search.

Returns:

list	A list of inventory item IDs.

GetNearestHiddenCoord

Get the nearest coordinate to the input agent that is not revealed to the agents player.

Parameters:

agent	The agent to test.

coordinate	The variable that will receive the resulting coordinate.

Returns:

boolean	TRUE if any coordinate was found.

GetNextWayPoint

Get the agent's next way point. If the agent is moving, this returns the way point the agent is moving toward. If the agent is not moving, this returns the current position of the agent.

Parameters:

agent	The agent to check.

Returns:

coordinate	If the agent is moving, this returns the way point the agent is moving toward. If the agent is not moving, this returns the current position of the agent.

GetParent

Get the supplied agent's parent. This is usually the Guild that spawned the agent. The will not return Reward Flag parents. Use GetFlag to return that type of parent.

Parameters:

agent	The agent to get the parent of.

Returns:

agent	The parent agent.

GetPlayerData

Get a value from a player.

Parameters:

agent	The agent who's player the data will be retrieved from.
string	The data type. This can be “gold” or “palace”

Returns:

integer	The player's total gold
agent	The player's palace

GetPlayerTeamNumber

Returns the team number of a player.

Parameters:

agent	Any agent of the player.

Returns:

integer	The team number of the player.

GetSpecialEvent1Script

Returns the GPL script function name registered for special event 1, using the current difficulty level.

Parameters: None

Returns:

string	The registered script function name.

GetSpecialEvent2Script

Returns the GPL script function name registered for special event 2, using the current difficulty level.

Parameters: None

Returns:

string	The registered script function name.

GetSpellAttribute

Get an attribute for a spell.

Parameters:

string	The spell name.
string	The attribute name, which can be: effector_duration timeout_duration character_level spell_rank

Returns:

integer	The attribute value.

GetUnitPlayerNumber

Returns the player who owns the unit.

Parameters:

agent	The agent to check.

Returns:

integer	The owning player number.

GetVictoryConditionIndex

Returns the victory condition type index. This is used by the gpl script function SetVictoryCondition to setup what type of victory condition is used for the scenario. See `Victory_Conditions.gpl` or `MX_Victory_Conditions.gpl`.

Parameters: None

Returns:

integer	The victory goal index.

GetVictoryConditionModifier

Returns the victory condition type modifier. This is used by the gpl script function SetVictoryCondition to setup what type of victory condition is used for the scenario. See `Victory_Conditions.gpl` or `MX_Victory_Conditions.gpl`.

Parameters: None

Returns:

integer	The victory goal modifier.

GetX

Get the X axis value from a coordinate.

Parameters:

coordinate	The coordinate to extract the value from.

Returns:

integer	The X axis coordinate member.

GetY

Get the Y axis value from a coordinate.

Parameters:

coordinate	The coordinate to extract the value from.

Returns:

integer	The Y axis coordinate member.

Halt

Stop GPL processing.

Parameters: None

Returns: Nothing.

HasAttribute

Return true if the agent has the supplied GPL attribute.

Parameters:

string	The attribute name.
agent	The agent to check.

Returns:

boolean	True if the attribute exists.

HasSpeedTrail

Returns true if the agent has speed trails.

Parameters:

agent	The agent to check.

Returns:

boolean	True if the agent has speed trails.

HasWayPoints

Returns true if the agent has way points assigned to control its movement.

Parameters:

agent	The agent to check.

Returns:

boolean	True if the agent has defined way points.

Hide

Order an agent to enter a building and become hidden from sight.

This command has an alias named **EnterBuilding**

Parameters:

agent	The agent to enter the building.
agent	The building agent.
integer (optional)	Modifier flags 0 — Walk to destination 1 — Teleport inside

Returns: Nothing.

IncrementStatCounter

Increments a statistical counter.

Parameters:

agent	The agent the the statistic refers to.
string	The statistic name. The only supported statistic to increment in this manner is "gutter".

Returns: Nothing.

InsideBuilding

If the supplied agent is inside a building, return true.

Parameters:

agent	The agent to check.

Returns:

boolean	True if the source agent is inside a building.

IsAdjacent

Test if the two agents are adjacent to each other. The agents must have a distance of 1 or less between their footprint areas on the path grid.

Parameters:

agent	Agent 1
agent	Agent 2

Return:

boolean	TRUE if the agents are adjacent.

IsEnteringBuilding

Return true if the agent has an order to enter a building.

Parameters:

agent	The agent to check.

Returns:

boolean	True if the agent has an order to enter a building.

IsHidden

If the supplied agent hidden from view, return true. An agent is hidden when inside a building.

Parameters:

agent	The agent to check.

Returns:

boolean	True if the source agent is hidden.

IsInventoryItem

Is the input item ID an inventory item?

Parameters:

integer or string	The item ID

Returns:

boolean	True if the item ID is an inventory item.

IsMessageFlagPresent

Returns true if a message flag, displaying the specified text ID is on the map.

Parameters:

integer or string	The string ID. If it is an integer, then the message comes from the indexed list of strings from the original game. If it is a string, the string is used as a key to look up the string in the quests string table.

Returns:

boolean	Returns true if the message flag is on the map and has not been dismissed by the player.

IsMoving

Test if the agent is moving.

Parameters:

agent	The agent to halt

Return:

boolean	TRUE if the agent is moving.

IsRunning

Test to see if a function has a thread associated with it. This will return true even if the thread is suspended.

Parameters:

function	The function to test.

Returns:

boolean	TRUE if the function has been assigned to a thread.

IsSpellAvailable

Returns true if the agent can cast the supplied spell at the current time. This tests to see if the agent knows the spell, has the proper experience level and optionally if the spell is timed out.

Parameters:

agent	The agent to check.
string	The spell list to check.
boolean (optional)	If true, the time out check will be skipped.

Returns:

boolean	True if the agent is able to cast the spell.

IsValidGamePiece

Test to see if the agent is attached to a valid game object.

Parameters:

agent	The agent to test.

Returns:

boolean	TRUE if the agent is associated with a valid game object.

KillThread

Stop the updating of a thread.

Parameters:

function	The thread to stop.

Returns: Nothing.

LastMusicTrack

Return the last music track played.

Parameters: None

Returns:

integer	The last music track played.

LearnSpell

Have an agent learn a spell.

Parameters:

agent	The agent that will learn the spell.
string	The spell name.
boolean (optional)	If true, show the spell in the interface, else hide it.

Returns: Nothing.

ListMember

Get an indexed member in a list. A list's index starts from 1 so with a list of N elements, the valid indexes are 1-N.

Parameters:

list	The list to index into.
integer	The index of the member, starting from 1.

Return:

any type	The list member.

ListObjects

Search for object on the map, in relation to an agent

Parameters:

agent	The agent to use as the search origin.
string	The unit type to match. Ex. "monster", "hero", "building", "rewardflag", "lair"
integer	The search radius, in pixels. Use -1 to search the entire map.
list	An existing list that will be filled with the results of the search.
integer	Optional modifier(s) See below.

Return:

integer	The list size.

The objects in the returned list are sorted nearest to furthest.

Modifiers are optional and are specified individually. Expressions have been defined for the modifiers, they are:

Modifiers:

#NoHiddenMap	Search all the object lists rather than just the visibility list for the searching agent.
#MyPlayer	Only objects that match the supplied agents player ID
#NotMyPlayer	Only objects that do NOT match the supplied agents player ID
#InsideOtherUnits	Can return units that are inside other units, ex. Heroes inside a building.
#RewardFlags	Find reward flags, this will ignore the passed in unitType and will search for reward flags. This is shorthand for passing in "rewardflag" as the unit type to match as well as adding **#InsideOtherUnits**.
#CheckTitles	This will filter the search by matching the 'title' attribute with a supplied comparison string. The comparison string must immediately follow this modifier. This cannot be used with the **#CheckSubTypes** modifier.
#CheckSubTypes	This will filter the search by matching the 'SubType' attribute with a supplied comparison string. The comparison string must immediately follow this modifier. This cannot be used with the **#CheckTitles** modifier.
#FindFirstMatch	This will only fill the list with the first unit found. This will not necessarily be the closest in distance.

Ex.

```
Get all the heroes on the map that are not owned by the supplied palace's
player and put them in the pre-declared list called NotMyHeroes.
$ListObjects (Palace, "Hero", -1, NotMyHeroes, #NoHiddenMap,
#InsideOtherUnits, #NotMyPlayer);
```

ListPalaces

Get a list of all the palaces on the map.

Parameters: None

Returns:

list	A list of palaces

Returns: Nothing

ListSize

Get the number of elements that are in the list.

Parameters:

list	The list to index into.

Return:

integer	The list size.

ListTitles

Take an input list of agents and return a list of the agents whose 'title' attribute matches the input. The 'title' of an agent is equivalent to the unit type. This is used to filter a large list of agents to find specified types.

Parameters:

list	A list of agents
string	The title to match.

Returns:

list	The agents who match the title name.

LocalChatMessage

Prints a status message to the UI. This is used primarily to display events such as a building being completed or upgraded.

Parameters:

integer	The player that the message is for
integer or string	The ID of the string to display. If it is an integer, then it is one of the indexed strings from the base game. If it is a string, then the string is the key name for the string to be displayed. See the documentation on adding text strings for more information.
agent (optional)	The agent that the message is associated with.

Returns: Nothing.

LocationOf

Get the coordinate of the agent.

Parameters:

agent	The agent to get the coordinate of.

Return:

coordinate	The coordinate of the agent.

LookupFunction

Find a GPL function by name.

Parameters:

string	The name of a GPL function.

Returns:

function	The function pointer, can be NullFunction if the function was not found.

MagicalAdjustAttribute

Changes an engine-side attribute value on an agent by a delta. This also adjust the attribute's Magical counterpart.

Parameters:

agent	The agent to change.
integer	The attribute's ID.
integer	The value to add to the attribute's current value

Returns: Nothing

MakeCoord

Make a coordinate from two integers

Parameters:

integer	The X axis value.
integer	The Y axis value.

Return:

coordinate	The coordinate.

MakeInventoryAttribute

Take an inventory ID as an input, which is a simple index, and turn it into an attribute ID.

Parameters:

integer	The inventory item ID.

Returns:

integer	The inventory item ID as an attribute ID.

MessageFlag

Places a message flag on the map, either at a specified coordinate, or on a unit.

Parameters:

coordinate or agent	Where to place the flag
integer or string	The string ID. If it is an integer, then the message comes from the indexed list of strings from the original game. If it is a string, the string is used as a key to look up the string in the quests string table.

Returns: Nothing.

MiniMapAnimation

Add an animation to the minimap. These are used as beacons to inform the player something of interest has occurred.

Parameters:

coordinate or agent	Where to place the beacon
string	Name of the beacon definition. The game provides, event_beacon, hero_death and treasure_find.

Returns: Nothing.

Move

Give an agent an order to move to a location.

Parameters:

agent	The agent to move
coordinate or agent	The destination of the move
string	Movement modifier. The input values can be:

add_waypoint	Append the move to the agent's movement queue, else it will replace any destination the agent is currently moving to
loop	Like **add_waypoint**, but the agent will loop back to the destination of the first movement location in the queue when this destination is reached.
avoid_vehicles	Avoid going near any other agent that is able to move. The distance is defined in the agent's buffer zone.
avoid_buildings	Avoid going near any agent's that are defined as buildings. The distance is defined in the agent's buffer zone.

	avoid_everything — Avoids both moving and non-moving agents.

Returns: Nothing.

NeutralTeamNumber

Returns the neutral team number.

Parameters: None

Returns:

integer	An available team number.

NewTeamNumber

Returns a team number that is not in use by any player.

Parameters: None

Returns:

integer	An available team number.

NewThread

Create a new thread from the input function. The function will be called every N milliseconds and will be passed in the remaining arguments. Note: GPL code execution is done on an frame based execution budget, so the threads execution interval will be approximate.

Parameters:

function	The function to call.
integer	Execution interval (approximate) in milliseconds.
...	Additional parameters of any type. They will be passed to the thread function.

Returns: Nothing.

NoDebugMessages

Turn off any debug messaging.

Parameters: None

Returns: Nothing.

NullAgent

Returns a null agent. This is useful for comparisons as well as setting agent member variables to an empty agent value.

Returns:

agent	The global null agent value.

Parent

Return the parent of the agent. An agent has a parent when it is inside a building, but can also have a parent that is a reward flag.

Parameters:

agent	The agent to retrieve the parent of.

Returns:

agentRef	The agent's parent, can be -1 if it does not have a parent.

PathCost

Returns the cost of a path between two coordinates with respect to a given unit. The last argument is optional, and if no unit is specified, the pathfinder makes an assumption about the "average" unit's cost of finding the path. If no path is found, the "can find path" parameter is set to false, otherwise it is set to true. It returns the cost of finding the path as a float.

Parameters:

coordinate	The starting coordinate.
coordinate	The destination coordinate.
string	Collision option **check_buildings** — Check against buildings. **check_static_units** — Check against static (unmoving) units. **check_all_units** — Check against all unit types.
boolean	Can Find Path, an output value that will be set to TRUE or FALSE as to whether the path was found.
agent	The agent to use to find the path.

Returns:

float	The path cost.

PerformAction

Tell the agent to perform a pre-defined action. See Actions for details on which actions are available.

Parameters:

agent	The agent to do the action
string	The name of the action
agent or coordinate	The agent or coordinate that is the target of the action.

Returns: Nothing.

PlaceRewardFlag

Place a reward flag on an agent.

Parameters:

agent	The agent to place the reward on.
integer	The player that is offering the reward.
integer	The amount of the reward.

Returns:

agent	The reward flag agent, if it was able to be created.

PlayMusic

Play the specified music track.

Parameters:

integer	The music track to play.

Returns: Nothing

PlaySound

Play a sound. Multiple parameter sets can be used, they are:

Parameters:

agent	The agent the sound is coming from.
string	The sound phase name. The sound description will be the one that is native to the agent type. See Sound Phases for a list of the valid sound phases.
float	(optional) distance modifier. This adjust the apparent range from the agent the

	sound is emanating from. The range is in world coordinates.

Parameters:

agent	The agent the sound is coming from.
string	Sound description name. See Sound Descriptions.
string	The sound phase name. See Sound Phases.
float	(optional) distance modifier. This adjust the apparent range from the agent the sound is emanating from. The range is in world coordinates.

Parameters:

string	Sound description name. See Sound Descriptions.
string	The sound phase name. See Sound Phases.
integer	(optional) The player that can hear the sound, all others will not hear it.

Returns: Nothing

RandomCoord

The function will return a random coordinate in a radius around the center point (either a unit or coord), at distance between the minimum (inner radius) and maximum. If only two parameters are supplied (center point and a radius), then the minimum is assumed to be zero. The min and max radii are rectangular, not circular.

Parameters:

agent or **coordinate**	The center point.
integer	(optional) Minimum distance from the center point. If not specified, 0 will be used.
integer	Maximum radius from the center point. Rectangular, not circular!

Return:

coordinate	A random coordinate.

RandomNumber

Return a random number from 0 up to and including the input value.

Parameters:

integer	Random number range, inclusive.

Return:

integer	A random number

RemoveAllBuildingLimits

Removes all limitations on all building types imposed by **SetBuildingLimit**, or by the player in the build tree editor

Parameters: None

Returns: Nothing.

RemoveAttribute

Remove a GPL attribute from the agent. It is ok to call this if the attribute does not exist on the agent.

Parameters:

agent	The agent to remove the attribute from.
string	The name of the attribute to remove.

Returns: Nothing

RemoveBuildingLimit

Remove any limitations on building type imposed by **SetBuildingLimit**, or by the player in the Build Tree Editor. Players can use the build tree editor in freestyle games only.

Parameters:

string	The building type to remove the limit for.

Returns: Nothing.

RemoveListMember

Removes an indexed member in a list. A list's index starts from 1 so with a list of N elements, the valid indexes are 1-N.

Parameters:

list	The list to index into.
integer	The index of the member, starting from 1.

Returns: Nothing.

RemoveTitles

Take an input list of agents and remove any agents whose 'title' attribute matches the input. The 'title' of an agent is equivalent to the unit type. The removed agents will be returned in another list. This is similar to ListTitles, except the source list is modified as well.

Parameters:

list	A list of agents to remove from.
string	The title to match.

Returns:

list	The agents who match the title name.

ReRandom

Set the random seed for the random number generator.

Parameters:

integer	The random seed value.

Returns: Nothing.

ResumeThread

Resume the updating of a thread.

Parameters:

function	The thread to resume.

Returns: Nothing.

RetrieveAgent

Get an agent reference by name. This is most often used to retrieve the “root” agent which serves as the container for information about the current quest.

Parameters:

string	The name of the agent.

Return:

agent	The named agent. Can be $NullAgent

RevealArea

Reveals the map to the specified agent's player. The revealed area is specified by a map point and a radius around that point.

Parameters:

agent	Any agent of the player that will see the map area.
coordinate	The map location to reveal
integer	The radius around the location to reveal. If the radius is -1, the whole map is revealed.

Returns: Nothing.

RevealWholeMap

Reveals the whole map to the player.

Parameters:

integer	The player number.

Returns: Nothing.

RevertUnitType

Changes the type of a unit back to its original type. This is only valid if ChangeUnitType has been called on the unit.

Parameters:

agent	The agent to change.

Returns: Nothing.

RunThread

Create a new thread from the input function. The function will be called once in N milliseconds and will be passed in the remaining arguments. Note: GPL code execution is done on an frame based execution budget, so the threads execution time will be approximate. If a thread using this function is already running, the timer will be reset to the new value.

Parameters:

function	The function to call.
integer	Execution time (approximate) in milliseconds.
...	Additional parameters of any type. They will be passed to the thread function.

Returns: Nothing.

RunThreadOnce

Create a new thread from the input function. The function will be called once in N milliseconds and will be passed in the remaining arguments. Note: GPL code execution is done on an frame based execution budget, so the threads execution time will be approximate. If a thread using this function is already running, the timer will be reset to the new value. The difference between this function and RunThread is that if the thread has already run and is in a suspended state, it will not be resumed.

Parameters:

function	The function to call.
integer	Execution time (approximate) in milliseconds.
...	Additional parameters of any type. They will be passed to the thread function.

Returns: Nothing.

SetAttribute

Set an engine-side attribute value on an agent.

Parameters:

agent	The agent to change.
integer	The attribute's ID.
integer	The attribute's value.

Returns: Nothing.

SetBuildingLimit

Limits all players to no more than the specified number of buildings for the specified type.

When the limit = 0 this has much the same effect as DisableUnitType.

Note that these limits are NOT enforced by SpawnUnit.

Parameters:

string	The building type to limit
integer	The limit value

Returns: Nothing.

SetDrawEffects

Sets a draw effect on the unit.

Parameters:

agent	The agent to update.
string	Palette name. Can be red, gray or green.
integer	Transparency amount 1 through 9.

Returns: Nothing.

SetEffectorDirection

Change the direction of an effect that is applied to an agent.

Parameters:

agent	The agent the effect is applied to.
string	The effect type to change.
integer	The direction of the effect. 0 – 31, with 0 being NORTH and increasing values going clockwise around the agent.

Returns: Nothing

SetHallFlag

Set the special monster effector if any player has this agents unit type selected. This is specific to the Hall Of Champions building.

Parameters:

agent	The agent, it must belong to the Monster player.

Returns: Nothing

SetPalace

Set the building that will be used as the player's Palace.

Parameters:

agent	The agent to use as the new palace
integer	The player ID.

Returns: Nothing

SetParent

Sets the parent of an agent. How the agent is associated with the parent is determined by the parent's type. If the parent agent is a building, usually a guild building, then the child agent becomes part of that guild. If the parent agent is a Reward Flag, then the child is chasing that flag. An agent can have both types of parents at the same time, a Guild and a Reward Flag.

Parameters:

agent	The agent to set the parent of.
agent	The parent agent.

Returns: Nothing

SetPlayerTeamNumber

Set the player to be on the specified team. All team members can 'see' the same map area.

Parameters:

agent	Any agent belonging to the player.
integer	The new team number.

Returns: Nothing.

SetShareReveal

Sets whether or not a player can see the same things that other players on their team can see. The team member must be one of the first four players.

Parameters:

integer	The player who will share the revealed map state with their team members.
boolean	True to share, false to not.

Returns: Nothing.

SetSpellUnitTimeout

Sets the duration for the spells effect, after which the spell will be destroyed.

Parameters:

agent	The spell agent.
integer	The timeout duration, in milliseconds.

Returns: Nothing.

SetStoppedMusicCallback

Set a callback that is done when a music track completes.

WARNING: The callback cannot modify anything that would change the game state, such as delete an agent or change an agent's values if the GPL script is to be run in a multiplayer game. This is because the callback will not be run on a machine that has the sound disabled or be called at different times if the music tracks are of different lengths on different client machines. Use this with care.

Parameters:

string	The function to name to call when the music completes.

Returns: Nothing.

SetThreadInterval

Change the interval at which a thread is called.

Parameters:

function	The function to call.
integer	Execution time (approximate) in milliseconds.

Returns: Nothing.

SetTreasuryValues

Sets the values for how often the player's treasury is checked and at what threshold a warning is given.

Parameters:

integer	The amount of time, in milliseconds, the treasury is checked.
integer	The amount of gold the treasury has to fall below before a warning is given.

Returns: Nothing.

SetUnitPlayerNumber

Change the unit's owning player to a new player.

Parameters:

agent	The agent to change.
integer	The new player owner's number.

Returns: Nothing.

ShowMinimap

Show or hide an agent in the minimap.

Parameters:

agent	The agent to change.
boolean	True to show the agent, false to hide it.

Returns: Nothing.

SpawnGraveyard

Create a new graveyard.

Parameters:

agent	The agent the graveyard is associated with. It must be a building.
string	The graveyard type name. Currently only “graveyard” is supported.
coordinate (optional)	The graveyard location. If not supplied, the graveyard will be placed near the associated building's exit point.

Ap
Returns: Nothing.

SpawnUnit

A given unit spawns another unit of a given type.

Parameters:

agent	The agent doing the spawning

string	Type name of the unit to spawn. If the spawning agent is a Lair, this can be overridden by the lair's data.
coordinate (optional)	Spawn location. If this is not specified, the spawning agent's exit point will be used.
integer (optional)	Player number who will own the new unit. If not specified, the owner of the spawning agent will own the new agent. This is often set to #Monster_Player when passing in a building the player owns to have a monster spawn near a player's building.
string (optional)	Extra options override: If specified, the Lair data is ignored and the specified unit type is used. maxhp: Forces the starting HP to maximum. For a building type, this means the first stage will be built. For a vehicle type, the Max HP will be set, else the type's initial HP will be used. clear: If specified, any units under the footprint of the new unit will be removed. This is useful when spawning buildings.
list (optional)	A list of Unit attribute ID/Value pairs.

Returns:

agent	The newly spawned agent.

SpecifyIntent

Sets the agent's "intent" string. This is the short text string that describes what the agent is thinking/doing.

Parameters:

agent	The agent set the intent on.
integer or string	If the parameter is an integer, then it is assumed to be the index in the built-in intent string table. If it is a string, then it is assumed to be a key value in the scenario's text table.

Returns: Nothing.

SpecifyName

Sets the agent's display name.

Parameters:

agent	The agent set the name on.
string	A key value in the scenario's text table.

Returns: Nothing.

Sqrt

Get the square root of the input value.

Parameters:

float	Input value

Return:

float	The square root. Note this is a low-precision square-root function.

StartEarthquakeSpell

Starts an earthquake effect, triggered by the supplied agent.

Parameters:

agent	The agent that the effect is triggered by.
integer	Minimum shake distance in pixels
integer	Maximum shake distance in pixels
integer (optional)	Duration in milliseconds. If not present or -1, then it is infinite. Use StopEarthquakeSpell to stop it.

Returns: Nothing.

StopEarthquakeSpell

Stop any earthquake spell the agent is triggering.

Parameters:

agent	The agent that has triggered the spell.

Returns: Nothing.

StopMoving

If the agent is moving, make it stop and clear its movement queue.

Parameters:

agent	The agent to halt

Returns: Nothing.

SuspendThread

Suspend the updating of a thread. Use ResumeThread to restart the thread update.

Parameters:

function	The thread to suspend.

Returns: Nothing.

TeleportToPoint

Teleport a unit from its current location toward a new location, by a specified amount. This is used for units that can 'jump' to a destination by teleporting. The location the unit ends up at will be determined by a 'best location' method.

Parameters:

agent	The agent to teleport.
integer	The distance to jump.
coordinate	The destination location.

Returns: Nothing.

TeleportToUnit

Teleport a unit from its current location toward a unit, by a specified amount. This is used for units that can 'jump' to a destination by teleporting. The location the unit ends up at will be determined by a 'best location' method.

Parameters:

agent	The agent to teleport.
integer	The distance to jump.
agent	The agent to teleport toward. If it is a building, the destination will be at the 'door' of the building.
integer	The spell casting range for the source unit. This is used to prevent the unit from getting too close to the destination if the destination is an enemy agent.

Returns: Nothing.

TransferInventoryItem

Transfer an inventory item from one agent to another.

Parameters:

integer	The item type ID.
agent	The agent giving the item.
agent	The agent receiving the item.

Returns: Nothing.

TurnOffSpeedTrail

Disables speed trails for the agent.

Parameters:

agent	The agent to remove the speed trails from.

Returns: Nothing.

TurnOnSpeedTrail

Enables speed trails for the agent.

Parameters:

agent	The agent to put speed trails on.
integer	The number of clones in the trail.
integer (optional)	The duration, in milliseconds, that the clone exists for.

Returns: Nothing.

Unhide

Has the agent exit any building it is in and become visible.

This command has an alias of **ExitBuilding**

Parameters:

agent	The agent to enter the building.
coord (optional)	Location to appear at.

Returns: Nothing

UnitsAreSamePlayer

Return true if the two agents belong to the same player.

Parameters:

agent	The agent to compare.
agent	The agent to compare.

Returns:

boolean	True if the two agents belong to the same player.

UnitIsLocalPlayer

Return true if the agent belongs to the local human player.

Parameters:

agent	The agent to compare.

Returns:

boolean	True if the agent belongs to the local human player.

UnitIsOffmapLair

Returns true if the agent is an off-map lair.

Parameters:

agent	The agent to check.

Returns:

boolean	True if the agent is an off-map lair.

UpgradeAgentAttributes

Copies the GPL attribute values from the agent's definition template into its local storage. This is usually called when a building upgrades to the next level.

Parameters:

agent	The agent to upgrade.

Returns: Nothing

ValidFunction

Test to see if the input is a valid function.

Parameters:

function	The function to test.

Returns:

boolean	TRUE if the input is a valid function reference.